Relatório de Implementação do Código

Sistema de Gestão de Ginásio • Fase 1

Autores: Fausto Henriques e Rodrigo Correia

1. Organização das classes

O código está organizado em classes que representam os elementos do ginásio e as operações necessárias para os gerir. A estrutura usa encapsulamento: os atributos são, em geral, privados (private) e são disponibilizados através de métodos de acesso (getters). As duas hierarquias de herança principais permitem separar os dados comuns dos dados específicos de cada tipo.

HIERARQUIA DOS SÓCIOS

```
Socio
  ├─ SocioIndividual
  └─ SocioEmpresa
```

HIERARQUIA DAS ATIVIDADES

```
Atividade
  ├─ AulaGrupo
  └─ SessaoPersonalTrainer
```

2. Classes relacionadas com os sócios

2.1. Socio

A classe Socio reúne os dados que são comuns a qualquer sócio: nome, identificador (id), endereço de correio eletrónico e desconto. O seu construtor recebe estes valores e os métodos getNome(), getId(), getEmail() e getDesconto() permitem consultar os atributos sem os tornar públicos.

Justificação da herança. Um sócio individual e um sócio de empresa partilham estes quatro elementos. Colocá-los na classe Socio evita repetir os mesmos atributos e métodos nas duas classes. Também permite que outras classes, como Marcacao, trabalhem com um objeto do tipo Socio sem precisarem de conhecer todos os pormenores do tipo concreto.

2.2. SocioIndividual

SocioIndividual herda nome, id, email e desconto de Socio e acrescenta o atributo anoNascimento. No construtor, super(nome, id, email, calcularDesconto(anoNascimento)) inicializa a parte herdada e envia para a classe pai o desconto calculado a partir do ano de nascimento.

O método calcularDesconto() calcula a idade usando o ano atual, através de Year.now().getValue(). Se a idade for inferior a 18 anos, atribui 10%; se for igual ou superior a 65 anos, atribui 15%; nos restantes casos, o desconto é 0%. O método showData() apresenta os dados herdados e o ano de nascimento.

Porque é uma classe-filha. O sócio individual é um tipo específico de sócio: mantém todos os dados comuns, mas necessita de guardar o ano de nascimento e de calcular o desconto de acordo com a idade.

2.3. SocioEmpresa

SocioEmpresa também herda os atributos comuns de Socio, acrescentando o atributo nif, que guarda o número de identificação fiscal da empresa como texto. O construtor utiliza super(nome, id, email, 0.2), pelo que o desconto atribuído é de 20%. O método showData() apresenta os dados gerais e o NIF.

Razão da especialização de SocioEmpresa. Esta classe representa um sócio empresa, pelo que partilha os dados gerais, mas precisa de guardar o NIF e aplicar uma regra de desconto própria. A herança reaproveita a estrutura comum, e permite a adição destas novas características.

3. Classes relacionadas com as atividades

3.1. Atividade

Atividade concentra os elementos que existem tanto numa aula de grupo como numa sessão de Personal Training: nome, instrutor responsável, data e hora (Tempo), duração em minutos e custo base. O construtor inicializa esses atributos e os métodos de acesso (getters) permitem consultar os valores.

Justificação da herança. As duas modalidades são atividades do ginásio e partilham informação (nome, responsável, data, duração e preço). Definir essa informação uma única vez reduz a duplicação e estabelece um tipo comum para o resto do programa.

3.2. AulaGrupo

AulaGrupo herda os dados gerais de Atividade e acrescenta modalidadeEnum e maxCap, que representam, respetivamente, a modalidade e a capacidade máxima de participantes. O construtor chama super(...) e calcula o custo base através de obterPrecoModalidade(). Os preços definidos são: Yoga — 8,00 €; Spinning — 9,00 €; Zumba — 8,50 €; CrossFit — 12,00 €. O método getMaxCap() permite o acesso ao valor da capacidade e o método showData() apresenta os dados da aula.

Porque é uma classe-filha. Uma aula de grupo precisa de todos os elementos comuns de uma atividade, mas tem características próprias: uma modalidade e um limite de participantes. Estas propriedades ficam em AulaGrupo, sem acrescentar à classe Atividade atributos que não são comuns a todas as atividades.

3.3. SessaoPersonalTrainer

SessaoPersonalTrainer herda os dados gerais de Atividade e acrescenta nivelEnum, que identifica o nível da sessão. O seu construtor usa super(...) e obtém o custo através de obterPrecoNivel(): Iniciação — 18,00 €; Intermédio — 22,00 €; Avançado — 28,00 €. O método showData() apresenta os dados comuns, o nível e o custo base.

Porque é uma classe-filha. A sessão personalizada é uma atividade, mas o preço depende do nível de treino e não de uma modalidade de aula de grupo. A classe filha permite implementar esta regra sem misturar a lógica de preços das duas categorias.

A separação entre AulaGrupo e SessaoPersonalTrainer é útil também no controlo das marcações: as aulas de grupo são limitadas pela capacidade indicada em maxCap, enquanto que uma sessão de Personal Training é considerada ocupada quando já existe uma marcação associada a essa sessão.

4. Classes de suporte e tipos enumerados

4.1. Instrutor

Instrutor guarda o nome, o género e a formação do mesmo. Os três atributos são privados e disponibilizados através de getters.

4.2. Tempo

Tempo guarda ano, mês, dia, hora e minutos. getDate() devolve a data e hora no formato AAAA-MM-DD HH:mm, getSimpleDate() devolve apenas a data e getMinutes() converte a hora em minutos desde o início do dia. Essa conversão facilita a comparação de intervalos de tempo. Na leitura de novas datas, é verificado o formato da data utilizando o método LocalDateTime.of(...) para validar a combinação de valores antes de criar o objeto Tempo.

4.3. Marcacao

Marcacao associa um objeto Socio a um objeto Atividade e guarda a data e hora, o custo final e o valor poupado. No construtor, a data é obtida a partir da atividade e o método calcCustoFinal() calcula o desconto: valor poupado = custo base × desconto; custo final = custo base − valor poupado. O método toString() prepara uma representação textual da marcação. Os getters permitem consultar o custo, a poupança, o sócio e a atividade.

Importância dos tipos-pai. Marcacao declara o seu campo como Socio (classe pai) e o campo da atividade como Atividade (classe pai). Assim, pode ser usada com qualquer uma das classes filhas sem criar versões diferentes de Marcacao para cada tipo de sócio ou de atividade.

4.4. ModalidadeEnum e NivelEnum

ModalidadeEnum define as modalidades Yoga, Spinning, Zumba e CrossFit. NivelEnum define Iniciação, Intermédio e Avançado. Os enums representam conjuntos fechados de opções e são usados pelos menus de seleção e pelos construtores, evitando que estes atributos dependam de texto livre.

5. Gestão e execução da aplicação

5.1. Ginasio — gestão dos dados e das operações

Ginasio é responsável pela gestão dos sócios, instrutores, atividades e marcações. Mantém ArrayLists separados para sócios individuais, sócios empresa, instrutores, aulas de grupo, sessões de Personal Training e marcações. O contador idCount atribui identificadores aos novos sócios, garantindo que não existem identificadores iguais. Ao ser construído, o objeto inicializa as listas, cria o Scanner para ler entradas da consola e chama carregarDadosDemo(), que adiciona dados de demonstração em memória.

Os métodos de registo recolhem os dados, criam os objetos correspondentes e acrescentam-nos às listas. Os métodos de seleção apresentam opções existentes. Os métodos de listagem mostram sócios, atividades e marcações; outros calculam resumos, identificam a atividade com mais marcações e somam os valores gastos e poupados por um sócio.

A classe também contêm funções para receber e validar a entrada: lerTexto() rejeita texto vazio e pode verificar um formato por expressão regular (usando um regex); lerInteiro() confirma que a entrada é um número inteiro e respeita um mínimo; lerData() verifica o formato da data e a validade da mesma. No registo de sócios, são verificados o formato do email e, no caso de uma empresa, o formato de nove algarismos do NIF (recorrendo ao uso do regex).

Antes de registar uma atividade, verificarSobrePosicoesInstrutor() compara o seu horário com as atividades já atribuídas ao mesmo instrutor. Antes de criar uma marcação, verificarSobrePosicoes() impede que o mesmo sócio tenha duas atividades sobrepostas no mesmo dia. Os métodos isAGFull() e isSPTFull() verificam, respetivamente, a capacidade de uma aula de grupo e se uma sessão de Personal Training já tem uma marcação.

Embora as listas de atividades sejam mantidas separadamente por tipo, o código também usa o tipo geral Atividade quando precisa de analisar ambas em conjunto. Por exemplo, verificarSobrePosicoesInstrutor() copia as aulas de grupo e as sessões de Personal Training para uma lista de Atividade, permitindo aplicar a mesma verificação aos dois tipos.

5.2. Main — entrada do programa e menu principal

Main contém o método main(), cria um objeto Ginasio e apresenta o menu principal. Cada opção encaminha a operação para o método adequado de Ginasio, como registar sócios, registar instrutores, criar atividades, listagens ou criar marcações. A função lerInteiro() do menu valida a escolha como número inteiro. Ao selecionar a opção de saída, o programa fecha os recursos Scanner e termina.

6. Funcionamento conjunto das classes

O fluxo principal pode ser resumido da seguinte forma:

- Main inicia a aplicação e cria Ginasio.
- Ginasio prepara as listas e carrega os dados de demonstração.
- O utilizador escolhe uma operação no menu; Ginasio recolhe e valida os dados necessários.

- São criados objetos do tipo específico — por exemplo, SocioIndividual ou AulaGrupo — e guardados nas listas correspondentes.
- Ao criar uma marcação, Marcacao recebe um Socio e uma Atividade, calcula o preço final e o valor poupado, e o objeto fica guardado na lista de marcações.
- As listagens e os resumos consultam os objetos guardados para apresentar os resultados.